

Unit 4: XML and AJAX

PL/SQL: Declaring PL/SQL Variables, Writing Executable Statements, Using SQL Statements Within a PL/SQL

XML Basics – Parser – DOM – XPath – XSL/XSLT – XQuery – DTD – Schema – Namespaces.

AJAX Basics – Request and Response- AJAX using XML- AJAX using Java Script and JQuery.

What is PL/SQL?

- PL/SQL is a *block-structured* language.
- A block lets you group logically related declarations and statements.

Feature	SQL	PL/SQL
Purpose	SQL is a single query that is used to perform DML and DDL operations.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
Nature	It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execution	Executes single statement.	Executes block of code
Use Case	Data retrieval, manipulation and definition(eg. SELECT, INSERT, UPDATE)	Mainly used to create an application.
Syntax	SQL statements only	SQL Statements combined with procedural logic
Data Handling	Performs actions directly on the database.	Can contain SQL inside its blocks and is used for more control over data handling

Key Features of PL/SQL

PL/SQL brings the benefits of procedural programming to the relational database world. Some of the most important features of PL/SQL include:

- **Block Structure:** PL/SQL can execute a number of queries in one block using single command.
- **Procedural Constructs:** One can create a PL/SQL unit such as procedures, functions, packages, triggers, and types, which are stored in the database for reuse by applications.
- **Error Handling:** PL/SQL provides a feature to handle the exception which occurs in PL/SQL block known as exception handling block.
- **Reusable Code:** Create stored procedures, functions, triggers, and packages, which can be executed repeatedly.
- **Performance:** Reduces network traffic by executing multiple SQL statements within a single block

PL/SQL Block structure

- ❖ PL/SQL is a block structure language
- ❖ A program is divided into logical blocks
- ❖ A block structure contains each object within a block is 'scope'

PL/SQL Block Structure

- **DECLARE (optional)**
 - Variables, cursors, user-defined exceptions
- **BEGIN (mandatory)**
 - SQL statements
 - PL/SQL statements
- **EXCEPTION (optional)**
 - Actions to perform when errors occur
- **END; (mandatory)**



Types of Block

- Anonymous Block
 - Are blocks without header
- Named Block
 - Blocks having headers
 - Can be either Subprogram or triggers
 - Can be compiled separately and stored permanently in an Oracle database, Ready to be executed

Anonymous Block

- It is a **block** of code **without a name**.
- It can be used anywhere in the program.
- The program is sent to the server engine for execution at runtime.

Characteristics of anonymous blocks:

- Unnamed block
- Not stored in the database
- Declared inline at the point in an application where it is executed
- Compiled each time the application is executed
- Passed to the PL/SQL engine for execution at run time
- Cannot be invoked or called because it does not have a name and does not exist after it is executed

Named block

- It is **block of code that is named.**
- **Program** is the block that can be called and take arguments.
- **Procedure** is a program that can perform an action.
- **Function** is a subprogram returns a value.
- **Package** is a group of procedure and function.

PL/SQL Block

PL/SQL block consists **three sections**.

1. A declaration section.
2. An executable section.
3. An exception handling section.

General syntax:

[DECLARE

Declaration of constants, variables, cursors and executions]

BEGIN

Executable PL/SQL and SQL statements

[Exception

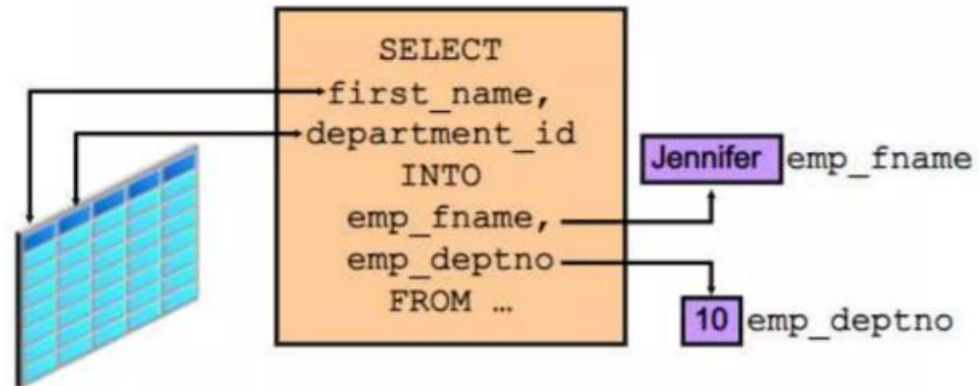
Action for error conditions]

END;

Use of Variables

Variables can be used for:

- Temporary storage of data
- Manipulation of stored values
- Reusability



Handling Variables in PL/SQL

Variables are:

- Declared and initialized in the declarative section
- Used and assigned new values in the executable section
- Passed as parameters to PL/SQL subprograms
- Used to hold the output of a PL/SQL subprogram

Syntax: variable_name datatype [NOT NULL] [:= initial_value];

Example:

DECLARE

v_studentName VARCHAR2(30); -- declare variable

v_marks NUMBER(3); -- declare variable

BEGIN

v_studentName := 'Adyasha'; -- assign value

v_marks := 95;

DBMS_OUTPUT.PUT_LINE('Name: ' || v_studentName);

DBMS_OUTPUT.PUT_LINE('Marks: ' || v_marks);

END;

Declaring & Initializing PL/SQL Variables

Syntax:

```
identifier [CONSTANT] datatype [NOT NULL]  
    [:= | DEFAULT expr];
```

Examples:

```
DECLARE  
    emp_hiredate      DATE;  
    emp_deptno        NUMBER(2) NOT NULL := 10;  
    location          VARCHAR2(13) := 'Atlanta';  
    c_comm             CONSTANT NUMBER := 1400;
```

Anchored declaration (% TYPE)

The %TYPE attribute

- Is used to declare a variable according to:
 - A database column definition
 - Another declared variable
- Is prefixed with:
 - The database table and column
 - The name of the declared variable

Advantages of Anchored Declaration

- Avoids **datatype mismatch errors**.
- Automatically adapts to **changes in table structure**.
- Makes code **shorter and more maintainable**.

- Anchored declaration means declaring a **variable** whose **datatype** is **automatically taken from a database column or another variable**.
- It uses the attributes:
 - **%TYPE** → Anchors to a single column or variable.
 - **%ROWTYPE** → Anchors to an entire row of a table or cursor.
- This helps in avoiding datatype mismatches and makes the code flexible.

%TYPE (Anchored to a Column or Variable)

- Variable takes its datatype from a **table column** or another variable.
- If the column datatype changes in the table, the PL/SQL variable automatically adjusts.
- Example:

```
DECLARE
    v_empName EMPLOYEES.FIRST_NAME%TYPE; -- datatype same as column FIRST_NAME
    v_salary  EMPLOYEES.SALARY%TYPE;      -- datatype same as column SALARY
BEGIN
    SELECT FIRST_NAME, SALARY
    INTO v_empName, v_salary
    FROM EMPLOYEES
    WHERE EMPLOYEE_ID = 100;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_empName || ', Salary: ' || v_salary);
END;
```


2. %ROWTYPE (Anchored to an Entire Row)

- Variable takes the datatype of an entire row of a table or cursor.
- Useful when we want to work with all the columns at once.
- **Example:**

```
DECLARE
    v_empRow EMPLOYEES%ROWTYPE; -- structure same as a row in EMPLOYEES
BEGIN
    SELECT * INTO v_empRow
    FROM EMPLOYEES
    WHERE EMPLOYEE_ID = 101;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_empRow.FIRST_NAME || ' ' || v_empRow.LAST_NAME
                          || ', Salary: ' || v_empRow.SALARY);
END;
```

Nested Anchoring

- The % TYPE attribute's use can be nested.
- Syntax

– DECLARE

-- **Source variable** v_commission

v_commission NUMBER(7,2);

--**Anchored Variable** v_total_commission

v_total_commission v_commission%TYPE;

--**Nested anchoring variable** v_net_commission

v_net_commission v_total_commission%TYPE;

NOT NULL Constraint for %TYPE

- If a source variable is declared with a NOT NULL constraint, the %TYPE declaration inherits the NOT NULL constraint from the source.

Different Variables

- Bind Variable
- Host Variable
- Substitution Variable

BIND VARIABLES:

- ❖ Bind variables are also known as **HOST VARIABLES**
- ❖ These variables are declared in the host SQL*Plus environment and are accessed by a **PL/SQL** block
- ❖ **Anonymous** blocks do not take any argument
- ❖ They can access host variables with a colon prefix(:)
- ❖ The syntax of a host variable declaration is
VARIABLE variablename datatype

Example:

SQL>VARIABLE double NUMBER

- ❖ When a numeric variables is declared with **VARIABLE** command, precision and scale values are not used
- ❖ If a **VARCHAR2**-type variable is declared length is not used
- ❖ A host variable's value can be printed in the SQL*Plus environment by using the **PRINT** command

A HOST VARIABLE IN A PL/SQL BLOCK

```
SQL>VARIABLE g_double NUMBER
```

```
SQL>DECLARE
```

```
    v_num NUMBER(2);
```

```
    BEGIN
```

```
        v_num:= 5;
```

```
        :g_double := v_num*2;
```

```
    END
```

```
    /
```

```
PL/SQL procedure successfully completed.
```

```
SQL>PRINT g_double
```

```
G_DOUBLE
```

```
.....
```

```
10
```

```
SQL
```

Substitution Variables in PL/SQL

- Substitution variables are used by prefixing a variable name with the ampersand(&) or double ampersand(&&).
- PL/SQL blocks can accept user input while execution.
- **Ampersand(&)** is used when you want to give a new value each time a substitution variable is encountered in the code.
- **Double ampersand (&&)** is used when you want the same value to be used for all occurrences of a variable in PL/SQL program.
- Substitution variables are available with limitations

Substitution Variables in PL/SQL

- Are used to get user input at run time
- Are referenced within a PL/SQL block with a preceding ampersand
- Are used to avoid hard coding values that can be obtained at run time

```
VARIABLE emp_salary NUMBER
SET AUTOPRINT ON
DECLARE
    empno NUMBER(6) := &empno;
BEGIN
    SELECT salary INTO :emp_salary
    FROM employees WHERE employee_id = empno;
END;
/
```

XML Basics

XML stands for **E**xtensible **M**arkup **L**anguage and is a text-based markup language derived from Standard Generalized Markup Language (SGML).

XML is a software- and hardware-independent tool for storing and transporting data.

- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive
- XML is a W3C Recommendation

XML tags identify the data and are used to store and organize the data, rather than specifying how to display it like HTML tags, which are used to display the data. XML is not going to replace HTML in the near future, but it introduces new possibilities by adopting many successful features of HTML.

There are three important characteristics of XML that make it useful in a variety of systems and solutions:

- **XML is extensible:** XML allows you to create your own self-descriptive tags, or language, that suits your application.
- **XML carries the data, does not present it:** XML allows you to store the data irrespective of how it will be presented.
- **XML is a public standard:** XML was developed by an organization called the World Wide Web Consortium (W3C) and is available as an open standard.

XML Features

1) XML separates data from HTML

With XML, data can be stored in separate XML files. This way you can focus on using HTML/CSS for display and layout, and be sure that changes in the underlying data will not require any changes to the HTML.

2) XML simplifies data sharing

In the real world, computer systems and databases contain data in incompatible formats.

XML data is stored in plain text format. This provides a software- and hardware-independent way of storing data.

This makes it much easier to create data that can be shared by different applications.

3) XML simplifies data transport

Exchanging data as XML greatly reduces this complexity, since the data can be read by different incompatible applications.

XML Features

4) XML simplifies Platform change

XML data is stored in text format. This makes it easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.

5) XML increases data availability

With XML, your data can be available to all kinds of "reading machines" (Handheld computers, voice machines, news feeds, etc.), and make it more available for blind people, or people with other disabilities.

6) XML can be used to create new internet languages

A lot of new Internet languages are created with XML.

Rules for writing XML

1. XML Documents Must Have a Root Element This element is "the parent" of all other elements.

```
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>
```

Example:

```
<friendsList>
  <friends>
    <name>Janak</name>
  </friends>
  <bestFriends>
    <name>Salina</name>
  </bestFriends>
</friendsList>
```

-FriendsList is root element

Rules for writing XML

2. The XML Prolog

The XML prolog is optional. If it exists, it must come first in the document.

```
<?xml version="1.0" encoding="UTF-8"?>
<friendlist>
  <to>Ram</to>
  <from>Shita</from>
  <body>Happy weekend</body>
</note>
```

Rules for writing XML

3. All XML Elements Must Have a Closing Tag

`<p>This is a paragraph.</p>`

4. XML Tags are Case Sensitive

XML tags are case sensitive. The tag `<Friend>` is different from the tag `<friend>`.

5. XML Elements Must be Properly Nested

Html: `<b><i>This text is bold and italic</b></i>`

XML `<b><i>This text is bold and italic</i></b>`

"Properly nested" simply means that since the `<i>` element is opened inside the `<b>` element, it must be closed inside the `<b>` element.

Rules for writing XML

6. XML Attribute Values Must Always be Quoted

```
<note date="12/11/2023">  
  <to>Ram</to>  
  <from>Sita</from>  
</note>
```

7. Entity References

Some characters have a special meaning in XML.

If you place a character like "<" inside an XML element, it will generate an error because the parser interprets it as the start of a new element.

Example: <student>marks < 100</student>

To avoid this error, replace the "<" character with an **entity reference**:

```
<student>marks &lt; 100</student>
```

XML Entity Reference

<	<	less than
>	>	greater than
&	&	ampersand
'	'	apostrophe
"	"	quotation mark

Rules for writing XML

8. Comments in XML

The syntax for writing comments in XML is similar to that of HTML:

Example: `<!-- This is a comment -->`

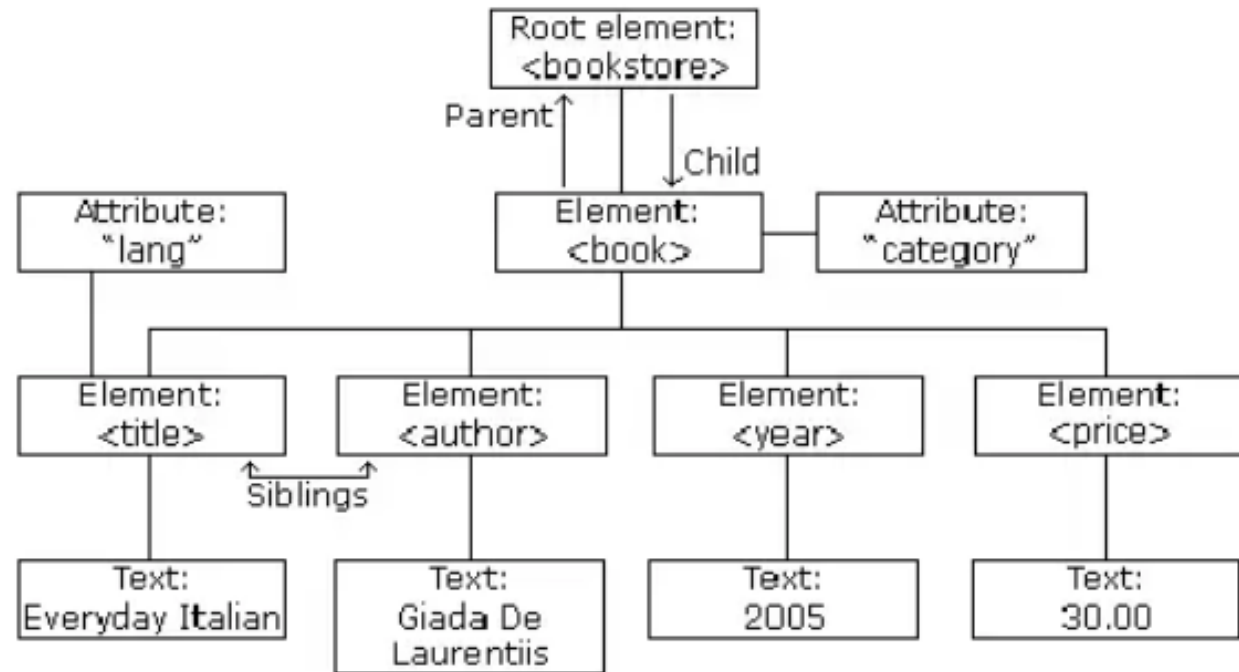
9. White-space is Preserved in XML

XML:	Hello Ram
HTML:	Hello Ram

```

<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>

```



XML Example

```
<?xml version="1.0" encoding="ISO-8859-1"?>  
<note>  
  <to>Ram</to>  
  <from>Shia</from>  
  <heading>Reminder</heading>  
  <body>Happy weekend!</body>  
</note>
```

- The first line is the XML declaration. It defines the XML version (1.0) and the encoding used (ISO-8859-1 = Latin-1/West European character set).
- The next line describes the root element of the document (like saying: "this document is a note"):
- The next 4 lines describe 4 child elements of the root (to, from, heading, and body).
- And finally the last line defines the end of the root element.

XML Elements

An XML element is everything from (including) the element's start tag to (including) the element's end tag.

An element can contain:

- text
- attributes
- other elements
- or a mix of the above

XML Elements

Example:

<friendsList>

 <friends>

 <name>Janak</name>

 </friends>

<bestFriends>

 <name>Salina</name>

</bestFriends>

<friendsList>

<name> has text content

<friendlist>, <bestFriends> has element content.

XML Attribute

XML Attributes Must be Quoted

Example:

<friendsList>

 <friends gender="male">

 <name>Janak</name>

 </friends>

<bestFriends>

 <name>Salina</name>

</bestFriends>

</friendsList>

Gender is attribute

XML Elements Vs Attribute

```
<person gender="female">  
  <firstname>Salina</firstname>  
  <lastname>Nagarkoti</lastname>  
</person>
```

```
<person>  
  <gender>female</gender>  
  <firstname>Salina</firstname>  
  <lastname>Nagarkoti</lastname>  
</person>
```

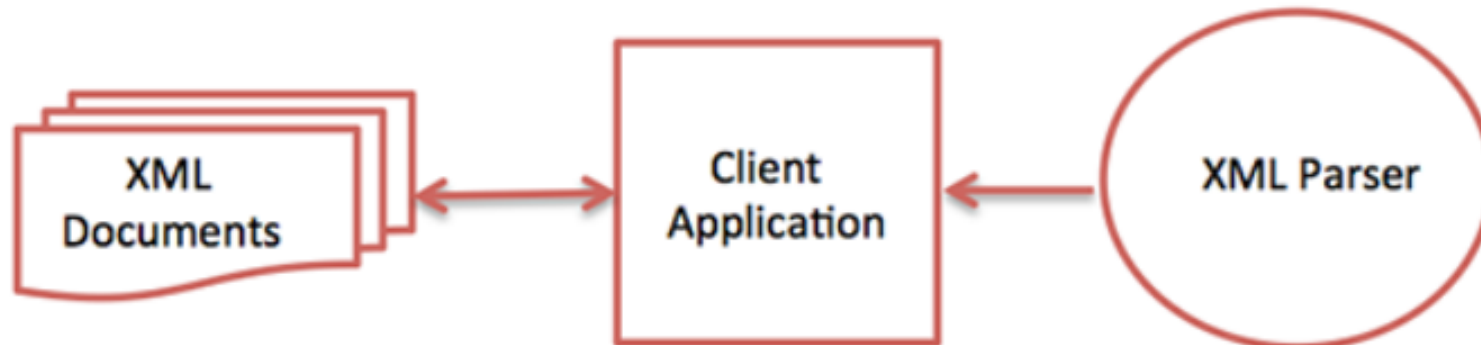
In the first example, gender is an attribute. In the last example, gender is an element. Both examples provide the same information.

There are no rules about when to use attributes or when to use elements in XML.

XML Parser

XML parser is a software library or a package that provides interface for client applications to work with XML documents. It checks for proper format of the XML document and may also validate the XML documents. Modern day browsers have built-in XML parsers.

Following diagram shows how XML parser interacts with XML document –



The goal of a parser is to transform XML into a readable code.

To ease the process of parsing, some commercial products are available that facilitate the breakdown of XML document and yield more reliable results.

XML Parser

- The [XML DOM \(Document Object Model\)](#) defines the properties and methods for accessing and editing XML.
- However, before an XML document can be accessed, it must be loaded into an XML DOM object.
- All modern browsers have a built-in XML parser that can convert text into an XML DOM object.

XML Parser

```
<html>
<body>

<p id="demo"></p>

<script>
var text, parser, xmlDoc;

text = "<bookstore><book>" +
"<title>Everyday Italian</title>" +
"<author>Giada De Laurentiis</author>" +
"<year>2005</year>" +
"</book></bookstore>";

parser = new DOMParser();
xmlDoc = parser.parseFromString(text,"text/xml");

document.getElementById("demo").innerHTML =
xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
</script>

</body>
</html>
```

Output:

Everyday Italian

XML DOM

What is the DOM?

- The Document Object Model (DOM) defines a standard for accessing and manipulating documents:
- *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."*
- The **HTML DOM** defines a standard way for accessing and manipulating HTML documents. It presents an HTML document as a tree-structure.
- The **XML DOM** defines a standard way for accessing and manipulating XML documents. It presents an XML document as a tree-structure.

XML DOM

Example:

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var parser, xmlDoc;
var text = "<bookstore><book>" +
"<title>Everyday Italian</title>" +
"<author>Giada De Laurentiis</author>" +
"<year>2005</year>" +
"</book></bookstore>";

parser = new DOMParser();
xmlDoc = parser.parseFromString(text,"text/xml");

document.getElementById("demo").innerHTML =
xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
</script>

</body>
</html>
```

XML DOM

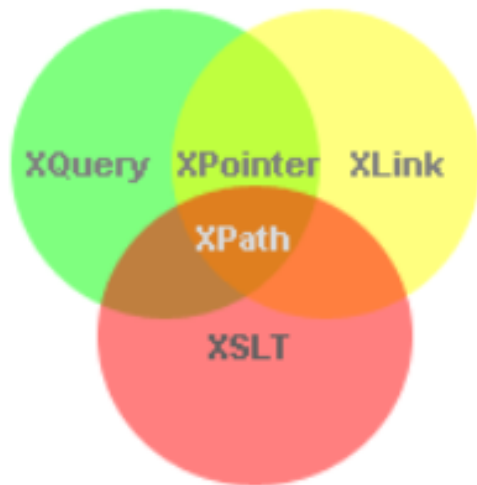
Feature	XML Parser (general)	DOM (specific parser)
Type	General approach to read XML	Tree-based model
Working	Reads XML (event or tree)	Builds full in-memory tree
Use case	Just reading/validating	Reading + modifying XML
Memory	Depends (streaming = low, tree = high)	High memory (loads whole XML)

XPath

What is XPath?

XPath is a major element in the XSLT standard.

XPath can be used to navigate through elements and attributes in an XML document.



- XPath is a syntax for defining parts of an XML document
- XPath uses path expressions to navigate in XML documents
- XPath contains a library of standard functions
- XPath is a major element in XSLT and in XQuery
- XPath is a W3C recommendation

XPath

XPath Path Expressions

- XPath uses path expressions to select nodes or node-sets in an XML document. These path expressions look very much like the expressions you see when you work with a traditional computer file system.
- XPath expressions can be used in JavaScript, Java, XML Schema, PHP, Python, C and C++, and lots of other languages.

XPath is Used in XSLT

- XPath is a major element in the XSLT standard.
- With XPath knowledge you will be able to take great advantage of XSL.

```
<?xml version="1.0" encoding="UTF-8"?>

<bookstore>

<book category="cooking">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2005</year>
  <price>30.00</price>
</book>

<book category="children">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>

<book category="web">
  <title lang="en">XQuery Kick Start</title>
  <author>James McGovern</author>
  <author>Per Bothner</author>
  <author>Kurt Cagle</author>
  <author>James Linn</author>
  <author>Vaidyanathan Nagarajan</author>
  <year>2003</year>
  <price>49.99</price>
</book>
```

```
<book category="web">
  <title lang="en">Learning XML</title>
  <author>Erik T. Ray</author>
  <year>2003</year>
  <price>39.95</price>
</book>

</bookstore>
```

XPath

In the table below we have listed some XPath expressions and the result of the expressions:

XPath Expression	Result
/bookstore/book[1]	Selects the first book element that is the child of the bookstore element
/bookstore/book[last()]	Selects the last book element that is the child of the bookstore element
/bookstore/book[last()-1]	Selects the last but one book element that is the child of the bookstore element
/bookstore/book[position()<3]	Selects the first two book elements that are children of the bookstore element
//title[@lang]	Selects all the title elements that have an attribute named lang
//title[@lang='en']	Selects all the title elements that have a "lang" attribute with a value of "en"
/bookstore/book[price>35.00]	Selects all the book elements of the bookstore element that have a price element with a value greater than 35.00
/bookstore/book[price>35.00]/title	Selects all the title elements of the book elements of the bookstore element that have a price element with a value greater than 35.00

XSLT

Displaying XML with XSLT

XSLT (eXtensible Stylesheet Language Transformations) is the recommended style sheet language for XML.

XSLT is far more sophisticated than CSS. With XSLT you can add/remove elements and attributes to or from the output file. You can also rearrange and sort elements, perform tests and make decisions about which elements to hide and display, and a lot more.

XSLT uses XPath to find information in an XML document.

XSLT

Belgian Waffles \$5.95 Two of our famous Belgian Waffles with plenty of real maple syrup 650 Strawberry
Belgian Waffles \$7.95 Light Belgian waffles covered with strawberries and whipped cream 900 Berry-Berry
Belgian Waffles \$8.95 Light Belgian waffles covered with an assortment of fresh berries and whipped cream 900
French Toast \$4.50 Thick slices made from our homemade sourdough bread 600 Homestyle Breakfast \$6.95 Two
eggs, bacon or sausage, toast, and our ever-popular hash browns 950

Belgian Waffles - \$5.95

Two of our famous Belgian Waffles with plenty of real maple syrup *(650 calories per serving)*

Strawberry Belgian Waffles - \$7.95

Light Belgian waffles covered with strawberries and whipped cream *(900 calories per serving)*

Berry-Berry Belgian Waffles - \$8.95

Light Belgian waffles covered with an assortment of fresh berries and whipped cream *(900 calories per serving)*

French Toast - \$4.50

Thick slices made from our homemade sourdough bread *(600 calories per serving)*

Homestyle Breakfast - \$6.95

Two eggs, bacon or sausage, toast, and our ever-popular hash browns *(950 calories per serving)*

XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<html xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<body style="font-family:Arial;font-size:12pt;background-color:#EEEEEE">
<xsl:for-each select="breakfast_menu/food">
  <div style="background-color:teal;color:white;padding:4px">
    <span style="font-weight:bold"><xsl:value-of select="name"/> - </span>
    <xsl:value-of select="price"/>
  </div>
  <div style="margin-left:20px;margin-bottom:1em;font-size:10pt">
    <p>
      <xsl:value-of select="description"/>
      <span style="font-style:italic"> (<xsl:value-of select="calories"/> calories per serving)</span>
    </p>
  </div>
</xsl:for-each>
</body>
</html>
```

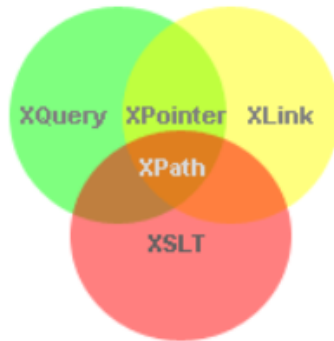
XQuery

What is XQuery?

XQuery is to XML what SQL is to databases.

XQuery was designed to query XML data.

What is XQuery?



- XQuery is **the** language for querying XML data
- XQuery for XML is like SQL for databases
- XQuery is built on XPath expressions
- XQuery is supported by all major databases
- XQuery is a W3C Recommendation

XQuery

XQuery Example

```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
order by $x/title
return $x/title
```


XML: DTD

- DTD stands for Document Type Definition.
- A DTD defines the structure and the legal elements and attributes of an XML document.
- It helps to validate XML

DTD are two types:

1. Internal
2. External

DTD: Internal declaration

If the DTD is declared inside the XML file, it must be wrapped inside the <!DOCTYPE> definition:

```
<?xml version="1.0"?>
<!DOCTYPE names [
  <!ELEMENT names (name,age)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT age (#PCDATA)>
]>
<names>
  <name>Nikesh</name>
  <age>22</age>
</names>
```

DTD: External Declaration

If the DTD is declared in an external file, the <!DOCTYPE> definition must contain a reference to the DTD file:

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE names SYSTEM "names.dtd">  
<names>  
<name>Nikesh</name>  
<age>20</age>  
</names>
```

The DOCTYPE declaration above contains a reference to a DTD file. The content of the DTD file is shown below.

```
<!DOCTYPE names  
[  
<!ELEMENT names (name,age)>  
<!ELEMENT name (#PCDATA)>  
<!ELEMENT age (#PCDATA)>  

```

XML: DTD

Internal DTD:

Syntax:

<!DOCTYPE rootelement DTD identifier

[

first declaration

second declaration

.

.

nth declaration

XML: DTD

When to Use a DTD?

- To interchanging data.
- To verify that the data you receive from the outside world is valid.
- Used a DTD to verify your own data.

XML: DTD

Important terms: DTD attribute default values and attribute types

(#PCDATA)- to represent parsable data

CDATA: to represent characters or symbols

ATTLIST: Attribute list

EMPTY: to represent the empty element

#IMPLIED: specifies that there is no default value for this attribute and that attribute is optional.

#REQUIRED: you don't provide default value

XML: DTD

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE names SYSTEM "names.dtd">
<names>
<name>Nikesh</name>
<age>20</age>
</names>
```

The DOCTYPE declaration above contains a reference to a DTD file. The content of the DTD file is shown below.

```
<!DOCTYPE names
[
<!ELEMENT names (name,age)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
]>
#PCDATA means parseable character data.
```

The DTD above is interpreted like this:

!DOCTYPE names - Defines that the root element of the document is names

!ELEMENT names - Defines that the names element must contain the elements: "name, age"

!ELEMENT name - Defines the name element to be of type "#PCDATA"

!ELEMENT age - Defines the age element to be of type "#PCDATA"

DTD: define attributes

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE names SYSTEM "names.dtd">
<names type="student">
  <name>Nikesh</name>
  <age>20</age>
</names>
```

The DOCTYPE declaration above contains a reference to a DTD file. The content of the DTD file is shown below.

```
<!DOCTYPE names
[
  <!ELEMENT names (name,age)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT age (#PCDATA)>
  <!!ATTLIST type ID #REQUIRED>
]>
```

The DTD above is interpreted like this:

!DOCTYPE names - Defines that the root element of the document is names

!ELEMENT names - Defines that the names element must contain the elements: "name, age"

!ELEMENT name - Defines the name element to be of type "#PCDATA"

!ELEMENT age - Defines the age element to be of type "#PCDATA"

<!!ATTLIST type defines the type attribute is REQUIRED

DTD: External Declaration

Example:

Data.xml

```
<!DOCTYPE data SYSTEM "data.dtd">
<data>
  <text>Hello, <em>World!</em></text>
  <script><![CDATA[alert("This is a CDATA section!");]]></script>
</data>
```

Data.dtd

```
<!ELEMENT data (text, script)>
<!ELEMENT text (#PCDATA | em)*>
<!ELEMENT script (#CDATA)>
```

-The <text> element's content model is defined as (#PCDATA | em)*. This means it can contain either parsed character data (#PCDATA) or the element. The * indicates that it can have zero or more occurrences of #PCDATA or . This allows us to include both plain text and elements within the <text> element.

-The <script> element's content model is defined as (#CDATA). This indicates that the content of the <script> element is treated as character data (#CDATA). The #CDATA section allows us to include raw text content, such as JavaScript code, without any special interpretation by the XML parser.

XML Schema

- An XML Schema describes the structure of an XML document, just like a DTD.
- An XML document with correct syntax is called "Well Formed".
- An XML document validated against an XML Schema is both "Well Formed" and "Valid".

XML Schema is an XML-based alternative to DTD:

```
<xs:element name="note">

  <xs:complexType>
    <xs:sequence>
      <xs:element name="to" type="xs:string"/>
      <xs:element name="from" type="xs:string"/>
      <xs:element name="heading" type="xs:string"/>
      <xs:element name="body" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

</xs:element>
```

XML Schema

The Schema above is interpreted like this:

- `<xs:element name="note">` defines the element called "note"
- `<xs:complexType>` the "note" element is a complex type
- `<xs:sequence>` the complex type is a sequence of elements
- `<xs:element name="to" type="xs:string">` the element "to" is of type string (text)
- `<xs:element name="from" type="xs:string">` the element "from" is of type string
- `<xs:element name="heading" type="xs:string">` the element "heading" is of type string
- `<xs:element name="body" type="xs:string">` the element "body" is of type string

XML Schemas are More Powerful than DTD

- XML Schemas are written in XML
- XML Schemas are extensible to additions
- XML Schemas support data types
- XML Schemas support namespaces

Why Use an XML Schema?

- With XML Schema, your XML files can carry a description of its own format.
- With XML Schema, independent groups of people can agree on a standard for interchanging data.
- With XML Schema, you can verify data.

XML Schemas Support Data Types

One of the greatest strengths of XML Schemas is the support for data types:

- It is easier to describe document content
- It is easier to define restrictions on data
- It is easier to validate the correctness of data
- It is easier to convert data between different data types

XML Schemas use XML Syntax

Another great strength about XML Schemas is that they are written in XML:

- You don't have to learn a new language
- You can use your XML editor to edit your Schema files
- You can use your XML parser to parse your Schema files
- You can manipulate your Schemas with the XML DOM
- You can transform your Schemas with XSLT

Name Space

What is an XML Namespace?

- An **XML Namespace** provides a way to **uniquely identify elements and attributes** in an XML document.
- XML documents often combine data from multiple sources.
- If two sources use the **same element names** (e.g., <table>, <name>), there will be confusion.
- A namespace attaches a **unique identifier (URI)** to distinguish them.
- Think of namespaces as **last names in real life**. Two people can be named “John”, but if one is “John Smith” and another is “John Brown”, you can tell them apart.

Syntax of Namespace

Namespaces are defined using the attribute:

`xmlns:prefix="URI"`

`xmlns` → keyword that declares namespace.

`prefix` → short name you assign.

`URI` → unique identifier (not necessarily an actual link, just a unique string).

Example:

```
<root xmlns:f="http://www.example.com/furniture"  
      xmlns:h="http://www.example.com/html">
```

```
<f:table>  
  <f:name>Dining Table</f:name>  
  <f:material>Wood</f:material>  
</f:table>
```

```
<h:table>  
  <h:tr>  
    <h:td>Row 1, Cell 1</h:td>  
    <h:td>Row 1, Cell 2</h:td>  
  </h:tr>  
</h:table>
```

```
</root>
```

- f:table refers to **furniture table**.
- h:table refers to **HTML table**.
- Prefixes f and h map to different namespaces (URIs).

Key Benefits of XML Namespaces

- Avoids element name conflicts.
- Makes XML documents modular and reusable.
- Helps when integrating XML from different standards (e.g., SVG + XHTML + MathML in one document).
- Ensures clarity and uniqueness.

- XML Namespace = a way to uniquely identify XML elements/attributes.
- Declared with xmlns:prefix="URI".
- Prevents naming conflicts.
- Can use prefixes or a default namespace.
- Attributes don't inherit default namespace automatically.

XML Namespaces - The xmlns Attribute:

- When using prefixes in XML, a **namespace** for the prefix must be defined.
- The namespace can be defined by an **xmlns** attribute in the start tag of an element.
- The namespace declaration has the following syntax. `xmlns:prefix="URI"`.
- Example:

```
<root>
```

```
<h:table xmlns:h="http://www.w3.org/TR/html4/">
```

```
<h:tr>
```

```
<h:td>Apples</h:td>
```

```
<h:td>Bananas</h:td>
```

```
</h:tr>
```

```
</h:table>
```

```
<f:table xmlns:f="https://www.w3schools.com/furniture">
```

```
<f:name>African Coffee Table</f:name>
```

```
<f:width>80</f:width>
```

```
<f:length>120</f:length>
```

```
</f:table>
```

```
</root>
```

The xmlns attribute in the first <table> element gives the h: prefix a qualified namespace.

The xmlns attribute in the second <table> element gives the f: prefix a qualified namespace.

When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace.

Namespaces can also be declared in the XML root element:

```
<root xmlns:h="http://www.w3.org/TR/html4/"  
      xmlns:f="https://www.w3schools.com/furniture">
```

Default Namespaces:

- Defining a default namespace for an element saves us from using prefixes in all the child elements. It has the following syntax:

`xmlns="namespaceURI"`

- This XML carries HTML table information:

```
<table xmlns="http://www.w3.org/TR/html4/">  
  <tr>  
    <td>Apples</td>  
    <td>Bananas</td>  
  </tr>  
</table>
```

XML Usage

A short list of XML usage says it all:

- XML can work behind the scene to simplify the creation of HTML documents for large web sites.
- XML can be used to exchange the information between organizations and systems.
- XML can be used for offloading and reloading of databases.
- XML can be used to store and arrange the data, which can customize your data handling needs.
- XML can easily be merged with style sheets to create almost any desired output.
- Virtually, any type of data can be expressed as an XML document.

How XML and HTML compare

XML	HTML
Used for storing data as structured information	Used for representing content
Strict validation	Loose validation
Defines encoded data	Defines display formatting
No predefined tags or semantics	Predefined tags and semantics
Tags are case-sensitive	Tags are not case-sensitive
New tags/elements can be added by the user	New tags/elements cannot be added by the user

What is AJAX

- AJAX is an acronym for **Asynchronous JavaScript and XML**. It is a group of inter-related technologies like JavaScript, DOM, XML, HTML/ XHTML, CSS, XMLHttpRequest etc.
- **AJAX allows you to send and receive data asynchronously without reloading the web page. So, it is fast.**
- AJAX allows you to send only important information to the server not the entire page. So only valuable data from the client side is routed to the server side. It makes your application interactive and faster.

Where it is used?

There are too many web applications running on the web that are using ajax technology like **gmail, facebook, twitter, google map, youtube** etc.

Famous web applications that make use of AJAX

Google Maps: A user can drag an entire map by using the mouse, rather than clicking on a button.

Google Suggest: As you type, Google offers suggestions. Use the arrow keys to navigate the results.

Gmail: Gmail is a webmail built on the idea that emails can be more intuitive, efficient, and useful.

Yahoo Maps (new): Now it's even easier and more fun to get where you're going!

BACK THEN...



AND YOU GET THIS ...

Web [Images](#) [Maps](#) [News](#) [Orkut](#) [Books](#) [Gmail](#) [more](#) ▼ [iGoogle](#) | [Search settings](#) | [Sign in](#)

Google

Search

About 17,200,000 results (0.11 seconds) [Advanced search](#)

Everything

- [Images](#)
- [Videos](#)
- [News](#)
- [Realtime](#)
- [Blogs](#)
- [More](#)

Ernakulam, Kerala
[Change location](#)

The web
[Pages from India](#)

Any time
[Latest](#)

News for roger federer

 [Photos of Roger Federer Out Celebrating Wife's Birthday in Miami](#) [48 minutes ago](#)
Tennis champ Roger Federer helps wife Mirka celebrate her 32nd birthday with a dinner at Miami's Prime Italian restaurant. (Pacific Coast News)more pics ...
[Zimbio](#) - 902 related articles

[I have many more years left - Federer](#) [SuperSport.com](#) - 1556 related articles

[Why Roger Federer won't win another major](#) [OregonLive.com](#) (blog)

[rogerfederer.com](#) [rogerfederer.com](#) [4 Feb 2011](#) ... Official site with results, schedule, galleries, forum and guestbook

Also try
[rafael nadal](#)
[roger federer girlfriend](#)

[See your ad here >](#)

THESE DAYS..



The screenshot shows the Google India homepage. At the top center is the Google logo with "India" written below it. Below the logo is a search bar containing the text "rog". A dropdown menu is open below the search bar, displaying a list of suggestions: "roger federer", "rog", "roger", "roger penrose", "roger that", "rogaine", "roger ebert", "rogue", "roger binny", and "roger moore". To the right of the search bar, there are links for "Advanced search" and "Language tools". Below the search bar, there are two buttons: "Google Search" and "I'm Feeling Lucky". On the left side of the search bar, there is a "Change" link. On the right side, there is a link for "Islam Punjabi".

Google

roger federer
rog
roger
roger penrose
roger that
rogaine
roger ebert
rogue
roger binny
roger moore

Advanced search
Language tools

Change

Google Search I'm Feeling Lucky

Islam Punjabi

GOOGLE SUGGEST

Google Suggest is using AJAX to create a very dynamic web interface: When you start typing in Google's search box, a JavaScript sends the letters off to a server and the server returns a list of suggestions.



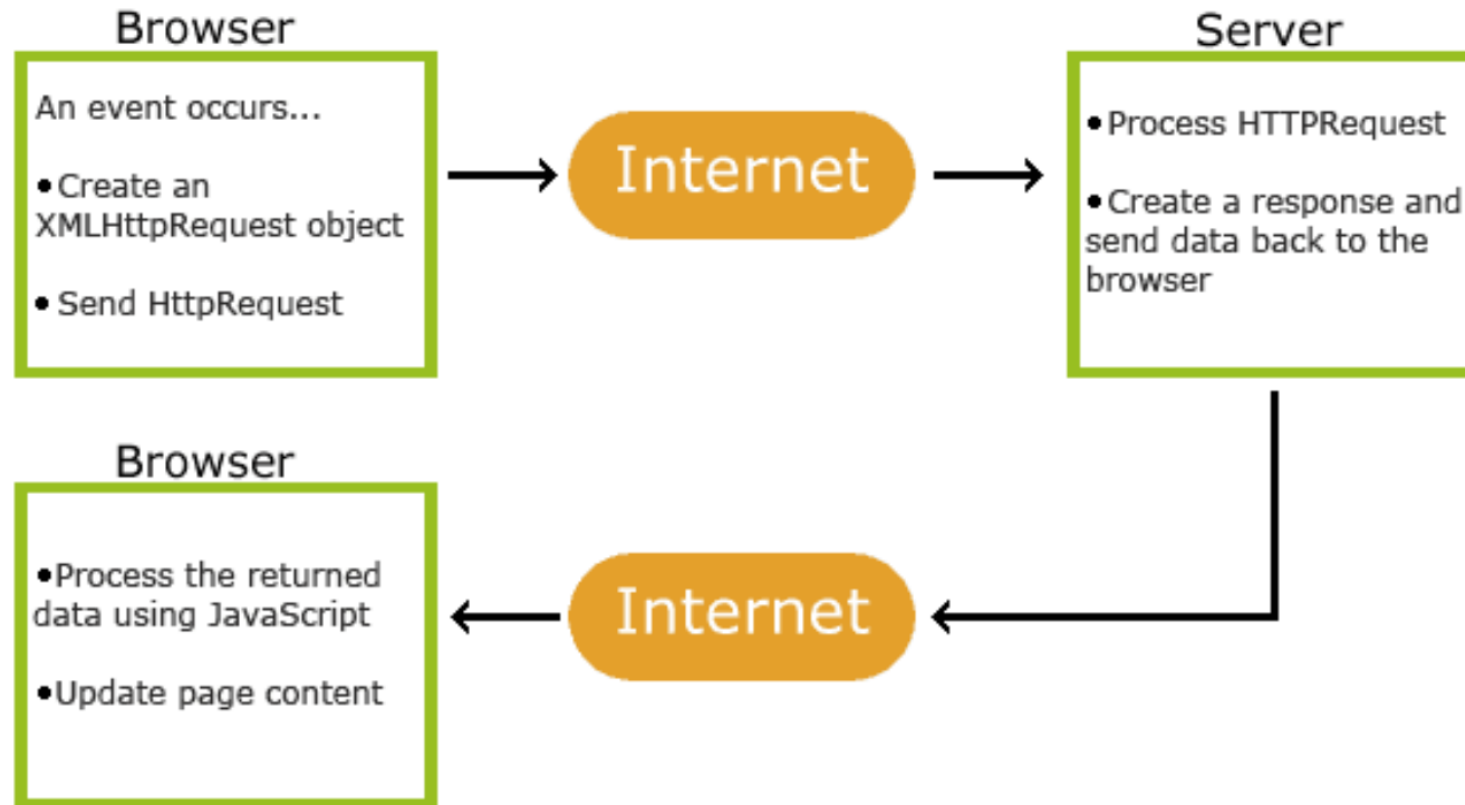
14 j		Recherche avancée
14 juillet belgique	2,520,000 résultats	Préférences
14 juillet wiki	275,000 résultats	Outils linguistiques
14 juillet syrie	381,000 résultats	
14 juillet aix les bains	469,000 résultats	rance
14 june	159,000,000 résultats	
14 juillet pétard	46,700 résultats	
14 juillet à deauville	196,000 résultats	e.com in English
14 juillet a toulouse	1,400,000 résultats	
14 juillet restaurant	800,000 résultats	
14 juillet wikipédia	311,000 résultats	
	fermer	

AJAX Technologies

As describe earlier, Ajax is not a technology but group of inter-related technologies. AJAX technologies includes:

1. **HTML/XHTML and CSS**-These technologies are used for displaying content and style. It is mainly used for presentation.
2. **DOM**- It is used for dynamic display and interaction with data.
3. **XML or JSON**- For carrying data to and from server. JSON (Javascript Object Notation) is like XML but short and faster than XML.
4. **XMLHttpRequest**- For asynchronous communication between client and server.
5. **JavaScript**- It is used to bring above technologies together. Independently, it is used mainly for client-side validation.

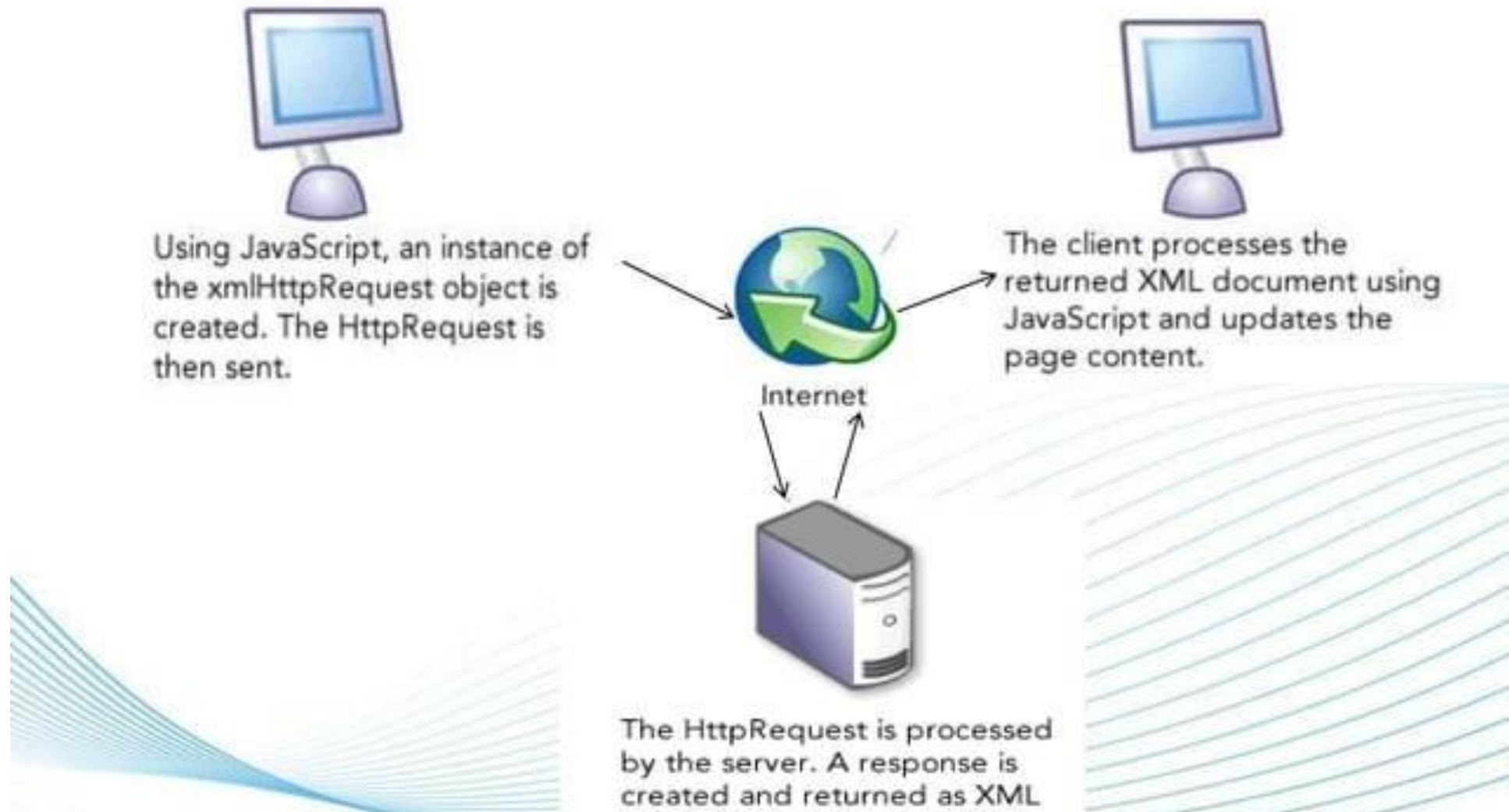
How AJAX Works



1. An event occurs in a web page (the page is loaded, a button is clicked)
2. An XMLHttpRequest object is created by JavaScript
3. The XMLHttpRequest object sends a request to a web server
4. The server processes the request
5. The server sends a response back to the web page
6. The response is read by JavaScript
7. Proper action (like page update) is performed by JavaScript

How AJAX Works

THE PROCESS CYCLE



How AJAX Works

- Update a web page without reloading the page
- Request data from a server - after the page has loaded
- Receive data from a server - after the page has loaded
- Send data to a server - in the background

How AJAX Works

```
<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h1>The XMLHttpRequest Object</h1>
<button type="button" onclick="loadDoc()">Change Content</button>
</div>

<script>
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML =
        this.responseText;
    }
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}
</script>

</body>
</html>
```

The HTML page contains a <div> section and a <button>.

The <div> section is used to display information from a server.

The <button> calls a function (if it is clicked).

The function requests data from a web server and displays it:

Result

The XMLHttpRequest Object



AJAX

AJAX is not a programming language.

AJAX is a technique for accessing web servers from a web page.

AJAX stands for Asynchronous JavaScript And XML.

XMLHttpRequest Object

Understanding XMLHttpRequest

An object of XMLHttpRequest is used for asynchronous communication between client and server.

It performs following operations:

- Sends data from the client in the background
- Receives the data from the server
- Updates the webpage without reloading it.

XMLHttpRequest Object Methods

Method	Description
<code>new XMLHttpRequest()</code>	Creates a new XMLHttpRequest object
<code>abort()</code>	Cancels the current request
<code>getAllResponseHeaders()</code>	Returns header information
<code>getResponseHeader()</code>	Returns specific header information
<code>open(method,url,async,user,psw)</code>	Specifies the request <i>method</i> : the request type GET or POST <i>url</i> : the file location <i>async</i> : true (asynchronous) or false (synchronous) <i>user</i> : optional user name <i>psw</i> : optional password
<code>send()</code>	Sends the request to the server Used for GET requests
<code>send(string)</code>	Sends the request to the server. Used for POST requests
<code>setRequestHeader()</code>	Adds a label/value pair to the header to be sent

XMLHttpRequest Object Properties

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the status-number of a request 200: "OK" 403: "Forbidden" 404: "Not Found" For a complete list go to the Http Messages Reference

AJAX Request

- The XMLHttpRequest object is used to exchange data with a server.
- **To send a request to a server, we use the `open()` and `send()` methods of the XMLHttpRequest object.**
- **Ex:**

```
xhttp.open("GET", "ajax_info.txt", true);  
xhttp.send();
```

Method	Description
<code>open(method, url, async)</code>	Specifies the type of request <i>method</i> : the type of request: GET or POST <i>url</i> : the server (file) location <i>async</i> : true (asynchronous) or false (synchronous)
<code>send()</code>	Sends the request to the server (used for GET)
<code>send(string)</code>	Sends the request to the server (used for POST)

AJAX Request

GET or POST?

- GET is simpler and faster than POST, and can be used in most cases.
- However, always use POST requests when:
 - A cached file is not an option (update a file or database on the server).
 - Sending a large amount of data to the server (POST has no size limitations).
 - Sending user input (which can contain unknown characters), POST is more robust and secure than GET.

AJAX Request

Ex: A simple GET request:

```
xhttp.open("GET", "demo_get.asp", true);  
xhttp.send();
```

The XMLHttpRequest Object

Request data

This content was requested using the GET method.

Requested at: 9/17/2025 6:00:59 AM

- In the example above, you may get a cached result. To avoid this, **add a unique ID to the URL .**

```
xhttp.open("GET", "demo_get.asp?t=" + Math.random(), true);  
xhttp.send();
```

- To send information with the GET method, add the information to the URL

```
xhttp.open("GET", "demo_get2.asp?fname=Henry&lname=Ford", true);  
xhttp.send();
```

AJAX Response

- The **readyState** property holds the status of the XMLHttpRequest.
- The **onreadystatechange** property defines a function to be executed when the readyState changes.
- The **status** property and the **statusText** property holds the status of the XMLHttpRequest object.

AJAX Response

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
status	200: "OK" 403: "Forbidden" 404: "Page not found" For a complete list go to the Http Messages Reference

AJAX Response

```
<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status ==
200) {

document.getElementById("demo").innerHT
ML =
    this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.txt", true);
    xhttp.send();
}
</script>
```

- The onreadystatechange function is called every time the readyState changes.
- When readyState is 4 and status is 200, the response is ready:

AJAX using XML

For the below example,

- When a user clicks on the "Get CD info" button above, the loadDoc() function is executed.
- The loadDoc() function creates an XMLHttpRequest object, adds the function to be executed when the server response is ready, and sends the request off to the server.
- When the server response is ready, an HTML table is built, nodes (elements) are extracted from the XML file, and it finally updates the element "demo" with the HTML table filled with XML data:

AJAX using XML

AJAX can be used for interactive communication with an XML file.

```
<!DOCTYPE html>
<html>
<style>
table,th,td {
    border : 1px solid black;
    border-collapse: collapse;
}
th,td {
    padding: 5px;
}
</style>
<body>

<h1>The XMLHttpRequest Object</h1>

<button type="button"
onclick="loadDoc()">Get my CD
collection</button>
<br><br>
<table id="demo"></table>
```

```
<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange =
function() {
    if (this.readyState == 4 &&
this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET",
"cd_catalog.xml", true);
xhttp.send();
}
```

```
function myFunction(xml) {
    var i;
    var xmlDoc = xml.responseXML;
    table="<tr><th>Artist</th><th>Title</th></tr>
>";
    var x =
xmlDoc.getElementsByTagName("CD");
    for (i = 0; i <x.length; i++) {
        table += "<tr><td>" +

x[i].getElementsByTagName("ARTIST")[0].child
dNodes[0].nodeValue +
        "</td><td>" +
x[i].getElementsByTagName("TITLE")[0].child
Nodes[0].nodeValue +
        "</td></tr>";
    }
    document.getElementById("demo").innerHT
ML = table;
}
</script>
</body>
</html>
```

AJAX using XML

For the previous example, Result:

The XMLHttpRequest Object

Get my CD collection

Artist	Title
Bob Dylan	Empire Burlesque
Bonnie Tyler	Hide your heart
Dolly Parton	Greatest Hits
Gary Moore	Still got the blues
Eros Ramazzotti	Eros
Bee Gees	One night only
Dr.Hook	Sylvias Mother
Rod Stewart	Maggie May
Andrea Bocelli	Romanza
Percy Sledge	When a man loves a woman
Savage Rose	Black angel
Many	1999 Grammy Nominees
Kenny Rogers	For the good times
Will Smith	Big Willie style
Van Morrison	Tupelo Honey

AJAX using Javascript and JQuery

- JQuery provides several methods for AJAX functionality.
- With the JQuery AJAX methods, you can request text, HTML, XML, or JSON from a remote server using both HTTP Get and HTTP Post - And you can load the external data directly into the selected HTML elements of your web page!

AJAX using Javascript and JQuery

jQuery - AJAX load() Method

- The jQuery load() method is a simple, but powerful AJAX method.
- The load() method loads data from a server and puts the returned data into the selected element.

Syntax:

```
$(selector).load(URL,data,callback);
```

- The required URL parameter specifies the URL you wish to load.
- The optional data parameter specifies a set of querystring key/value pairs to send along with the request.
- The optional callback parameter is the name of a function to be executed after the load() method is completed.

AJAX using Javascript and JQuery

- example loads the content of the file "demo_test.txt" into a specific <div> element:

`$("#div1").load("demo_test.txt");`

- The optional callback parameter specifies a callback function to run when the load() method is completed. The callback function can have different parameters:
 - **responseTxt** - contains the resulting content if the call succeeds
 - **statusTxt** - contains the status of the call
 - **xhr** - contains the XMLHttpRequest object

AJAX using Javascript and JQuery

Example displays an alert box after the load() method completes. If the load() method has succeeded, it displays "External content loaded successfully!", and if it fails it displays an error message:

```
$("#button").click(function(){
    $("#div1").load("demo_test.txt", function(responseTxt, statusTxt, xhr){
        if(statusTxt == "success")
            alert("External content loaded successfully!");
        if(statusTxt == "error")
            alert("Error: " + xhr.status + ": " + xhr.statusText);
    });
});
```

AJAX using Javascript and JQuery

jQuery - AJAX get() and post() Methods

- Two commonly used methods for a request-response between a client and server are: GET and POST.
 - **GET** - Requests data from a specified resource
 - **POST** - Submits data to be processed to a specified resource
- GET is basically used for just getting (retrieving) some data from the server.
- POST can also be used to get some data from the server. However, the POST method NEVER caches data, and is often used to send data along with the request.

Note: The GET method may return cached data.

AJAX using Javascript and JQuery

jQuery - AJAX get() and post() Methods

1. jQuery \$.get() Method

- The \$.get() method requests data from the server with an HTTP GET request.

Syntax: *\$.get(URL, callback);*

- The required URL parameter specifies the URL you wish to request.
- The optional callback parameter is the name of a function to be executed if the request succeeds.

AJAX using Javascript and JQuery

jQuery - AJAX get() and post() Methods

1. jQuery \$.get() Method

- Ex:

```
$("#button").click(function(){  
    $.get("demo_test.asp", function(data, status){  
        alert("Data: " + data + "\nStatus: " +  
status);  
    });  
});
```

- The first parameter of `$.get()` is the URL wish to request ("demo_test.asp").
- The second parameter is a callback function. The first callback parameter holds the content of the page requested, and the second callback parameter holds the status of the request.

AJAX using Javascript and JQuery

jQuery - AJAX get() and post() Methods

2. jQuery \$.post() Method

- The \$.post() method requests data from the server using an HTTP POST request.

Syntax: *\$.post(URL,data,callback);*

- The required URL parameter specifies the URL you wish to request.
- The optional data parameter specifies some data to send along with the request.
- The optional callback parameter is the name of a function to be executed if the request succeeds.

AJAX using Javascript and JQuery

jQuery - AJAX get() and post() Methods

2. jQuery \$.post() Method

- Ex:

```
$("button").click(function(){  
    $.post("demo_test_post.asp",  
    {  
        name: "Donald Duck",  
        city: "Duckburg"  
    },  
    function(data, status){  
        alert("Data: " + data + "\nStatus: " + status);  
    });  
});
```

- The first parameter of **\$.post()** is the URL we wish to request ("demo_test_post.asp").
- Then we pass in some data to send along with the request (name and city).
- The ASP script in "demo_test_post.asp" reads the parameters, processes them, and returns a result.
- The third parameter is a callback function. The first callback parameter holds the content of the page requested, and the second callback parameter holds the status of the request.

BENEFITS OF USING AJAX

- Helps to build fast, dynamic websites.
- Ajax allows to perform processing on client computer (in JavaScript) with data taken from the server thereby reducing server load by moving a part of server functionality to client side.
- Ajax can selectively modify a part of a page displayed by the browser, and update it without the need to reload the whole document with all images, menus etc. This bridges the gap between desktop and web applications.
- Saves bandwidth by only transmitting new information
- Creates possibility of entirely new types of user interfaces not possible in traditional model.

AJAX DISADVANTAGES

- Poor compatibility with very old or obscure browsers, and many mobile devices.
- Limited Capabilities like multimedia, interaction with web-cams and printers, local data storage and real time graphics.
- Not everyone have JavaScript enabled. If JavaScript is not activated, Ajax can't works. The user must be asked to set JavaScript from within options of the browser, with the "noscript" tag.
- Too much code makes the browser slow.

- The back button problem. People think that when they press back button, they will return to the last change they made, but in AJAX this doesn't hold.
- Possible network latency problems. People should be given feedback about the processing.
- Does not run on all browsers.

